

Pipeline guide

Stress-testing energy system designs against future climate (C2E, SSP4.5)

Yassir Lahlou | code version v16 | June 15, 2026

This document explains what the code does and what each file is for. It is meant to be read alongside the repository. The thesis question is simple to state. Energy system designs in the literature were optimised on past weather. Do they still work under future climate. We take a published design with its capacities fixed, swap in future weather from the Climate2Energy (C2E) dataset, re-run the operation of the system, and measure two things: how much demand goes unserved, and what the emissions are. The first published design we test is Gotske et al. (2024), a sector-coupled European net-zero system. We later compare against a system designed directly on the C2E data.

1. Folder structure

thesis-climate-energy/

src/	all the Python code
config.py	one place for all settings and file paths
mapping.py	country to network-bus lookup
inspect_c2e.py	prints what is inside the C2E files
c2e_loader.py	reads C2E files onto the network time grid
cf_methods.py	the two climate methods for wind and solar
heat_cooling.py	climate change applied to heating, cooling, hydro
build_modified_network.py	builds one future-weather network
dispatch.py	runs the operation and reads out the results
reporting.py	makes the figures, tables and Word reports
run_pipeline.py	the main script that runs everything
audit_real_data.py	pre-run checks on the real data
preflight_check.py	quick environment checks before a run
test_co2_mechanism.py	automated test of the carbon mechanism
test_v16_demand.py	automated test of the demand and hydro methods
_make_synthetic_selftest.py	builds fake data to test the pipeline offline
data/	
co2_totals.csv	1990 emissions reference (from Gotske)
docs/	written documentation (including this guide)
cloud/	notes and scripts for running on a cloud VM
output/	created when you run; all results go here
C2E/	the Climate2Energy data files (you provide these)
venv/	the Python environment and the network file

The data and results folders are not the code. You provide the C2E folder. The output folder is created automatically on the first run, and every run writes a timestamped sub-folder inside it.

2. How a run flows

One command does everything: `python -m src.run_pipeline`. Reading the steps below makes the rest of this document easier to follow, because it shows how the files connect.

- **Step 1, the reference.** The original Gotske network is loaded and run once on its own 2015 weather. This is the baseline that every future result is compared against. The carbon price is read from this network at the same time (explained in section 4). The reference is cached, so it is not re-solved for each method.
- **Step 2, for each method (qdm and direct).** A future-weather copy of the network is built

(`build_modified_network.py`), which calls the wind and solar methods (`cf_methods.py`) and the demand and hydro methods (`heat_cooling.py`). That copy is then run (`dispatch.py`), and the results are read out.

- **Step 3, reporting.** For each method, figures, tables and a Word report are written (`reporting.py`). A final comparison document puts the methods side by side against the reference.

3. The two methods (what they mean)

The future weather comes from a climate model, and the network was built on a different historical dataset (ERA5). The two never line up exactly, so there are two honest ways to bring the future in. Both are run, and the gap between them brackets the answer.

- **qdm (the primary method).** The network keeps its own 2015 order of the weather, and each variable is reshaped piece by piece to match the projected change. The technical name is Quantile Delta Mapping (Cannon et al. 2015). In plain terms, calm hours and windy hours are each adjusted by their own change rather than by one average, so changes in the extremes are kept. This isolates the climate change signal and removes the difference between the two datasets.
- **direct.** The system experiences the C2E future year as it is, with the raw values dropped in. This is the same idea Gotske themselves use when they test a design on a different historical weather year, extended here to a future year. It carries the full future world, including the modelling differences between C2E and the network, so it is the harder stress test of the two.

A quantile is just a position in the sorted data. If you line up a year of hourly values from lowest to highest, the value one tenth of the way along is a low-output hour, and the value nine tenths along is a high-output hour. Working per quantile means treating those two kinds of hour separately.

4. The carbon treatment (done the Gotske way)

When Gotske run the operation of a fixed system, they do not keep a hard net-zero limit. They put a price on emissions instead, and that price is the shadow price of the carbon limit taken from the original capacity optimisation. It is stored inside the network file and is about 468 euros per tonne. The pipeline reads that number from the file and applies it as a carbon price, with the hard limit removed. This is checked in their published config (the settings `custom_co2_price` false and `add_co2_lim` false). A hard limit in the operation step instead would distort the result, which is what an earlier version of our code did.

5. What each file does

Settings and shared lookups

`config.py` is the single place for every setting and file path. The weather year, the C2E period (2042 or 2099), which methods to run, the carbon settings, the load-shedding cost, and the names of the C2E files all live here. Every other file imports from it, so a run is reproducible. If something needs changing, it is almost always changed here.

`mapping.py` holds the lookup between countries and the network buses. The Gotske network has 37 regions with names like DE0. This file says which country each region belongs to, so that C2E country data can be matched to the right part of the network. Multi-region countries share their national data, and Luxembourg is represented by Belgium.

Reading the C2E data

`inspect_c2e.py` prints what is actually inside the C2E files: the columns, the date range, the resolution and the country list. It is the first thing to run when you get a new set of files, because it confirms the files are what you think they are. It does not change anything.

`c2e_loader.py` reads a C2E CSV and puts it onto the network time grid. It handles the file layout automatically and resamples to the network resolution. It does not force the dates to match exactly, because the C2E years and the network year are different calendar years; the series are matched by position, which is valid because both cover one full year at the same resolution.

Applying the climate change

`cf_methods.py` contains the wind and solar methods. It applies either qdm or direct to a capacity-factor series and returns the modified series. This is where the quantile mapping is actually computed.

`heat_cooling.py` applies the climate change to the demand side and to hydro. There are three parts.

- **Heating.** The network already has a heating demand. Only the change from C2E is applied, never the raw C2E numbers, so the level the network was built with is preserved and only the climate effect comes in. Under qdm this is a daily change worked out per quantile, so cold extreme days and mild days change differently. Under direct the future shape is transplanted with its annual level anchored to the original.
- **Cooling.** The network has no separate cooling demand, but the historical electricity load already contains today's cooling. The code takes the C2E baseline cooling, which is an estimate of that embedded cooling, subtracts it, and then adds the future cooling. Under qdm the future cooling is added as a typical-year pattern. Under direct the raw future cooling is added, so a hot spell and its matching low-wind hour line up. This replaces an earlier, weaker design that simply added cooling on top and double counted it.
- **Hydro.** Reservoir inflow and run-of-river availability are changed the same way as the other variables. Countries that the C2E hydro files do not cover are left unchanged and are listed in the log.

There is also an optional heat-pump efficiency step. It is left off, because the C2E files have no temperature series for it to use, and for several countries the C2E heating already includes the efficiency effect, so changing it again would double count.

`build_modified_network.py` ties the above together for one method. It loads the original network, loads the C2E data for the baseline and future periods, calls the wind and solar method and the demand and hydro methods, and saves the modified network together with tables describing exactly what was changed.

Running the system and reading the results

`dispatch.py` is the heart of the operation step. It prepares a network the Gotske way (it fixes the built capacities, adds the load-shedding option at the right buses, removes the hard carbon limit and applies the carbon price), then solves the operation, then reads out a detailed set of results. The results include unserved energy, resource adequacy, emissions counted from every source that touches the carbon accounting (including a cross-check that the numbers close), the generation mix, curtailment, storage use and more.

`reporting.py` turns the results into deliverables. For each method it writes organised folders of figures and tables and a Word report. It also writes a comparison document that places the methods next to each other and next to the reference. Every figure has a backing CSV so the numbers can be checked.

`run_pipeline.py` is the main script. It runs the reference once, loops over the methods, and calls the builder, the dispatch and the reporting in order, then writes the comparison. This is the file you run.

Checks and tests

`audit_real_data.py` runs before the expensive solve and checks the real data without solving anything. It confirms the C2E countries match the network, that heating units make sense, that hydro maps correctly, that the turbine files are present, that the carbon accounting parts are set up as Gotske require, and that cooling levels are sensible. It prints a clear verdict for each check.

`preflight_check.py` is a quick environment check (that the solver, the network file and the settings are in place) before a real run.

`test_co2_mechanism.py` is an automated test of the carbon mechanism on a tiny network. It confirms that the capacity freeze keeps the right components free, that the load-shedding option is set up correctly, that emitters pay the carbon price and direct air capture is credited, and that the emissions accounting closes exactly.

`test_v16_demand.py` is an automated test of the heating, cooling and hydro methods. It confirms the mathematical behaviour directly, so that any problem points at the transform rather than the solver. For example, it checks that the direct heating method scales the annual heat by exactly the right ratio and keeps the daily shape, and that the cooling subtraction and addition cancel out when the future equals the baseline.

`make_synthetic_selftest.py` builds a small fake network and fake C2E files so the whole pipeline can be run offline, without the real data. It is only a test fixture and is not part of any thesis result.

6. What changed since v14

The earlier version the supervisor saw used one method (the old delta method), with no heating, no cooling, and no hydro. Versions v15 and v16 brought the pipeline in line with what the thesis needs and fixed several real problems. The main changes, grouped, are below.

Made the operation faithful to Gotske (v15)

The code was checked line by line against Gotske's published code, not just their paper. Several real fixes followed.

- **Capacity freeze.** The earlier code froze every component, which silently capped emissions and capped the fuel supply rate during stressed hours. Gotske freeze only the costed technologies and leave the carbon accounting and the fuel supply free. This is now matched exactly.
- **Load shedding.** The cost, the location and the sizing were all wrong. They are now set to Gotske's values: a very high price, placed only at the electricity and heat buses, and unlimited in size so it is purely a last resort.
- **Direct air capture.** It was not being credited the carbon price. Gotske credit it, and now so do we.
- **Emissions accounting.** The earlier metric missed the fixed emission flows that sit on the carbon accounting bus (for example oil and process emissions). Counting them changes the headline emissions number a lot. A self cross-check was added so the emissions are counted two independent ways and must agree.
- **Carbon price.** It is now read directly from the design network file (about 468 euros per tonne) rather than re-derived, which an earlier run had got wrong.
- **A reporting bug** where the reference run overwrote the modified tables was fixed. Each table now exists twice, once for the modified run and once for the reference.

Redesigned the methods (v16)

- **The old delta method was removed.** It removed the dataset difference like qdm but kept only the average change, so it had no advantage over qdm and no transparency advantage over direct. The two methods are now qdm and direct, each a coherent world.
- **Heating** now uses a daily change under qdm and a shape transplant under direct, instead of a coarse monthly factor.
- **Cooling** was redesigned. The earlier version added cooling on top of loads that already contained it, which double counted. It now extracts the embedded historical cooling first and then adds the future cooling.
- **Hydro** was made consistent with the two methods.
- **The wind turbine** was selected to match the network’s assumptions by specific power (the SWT120_3600 model), which matters for the direct method.
- **The scenario** is confirmed as SSP4.5 across 2015 to 2100, matching the project description.

Other

New output tables were added for resource adequacy on Gotske’s own basis, a cost breakdown, a seasonal summary, stress indicators and the demand changes. New automated tests were added for the carbon mechanism and the demand methods. A real bug in the cooling code that would have crashed every run was found and fixed during testing.

Main references behind the choices: Gotske et al. (2024) for the system and the carbon method; Cannon et al. (2015) for quantile delta mapping; Wohland et al. (2025) for the Climate2Energy dataset; the IEA cooling report (2018) for the cooling cross-check; and When2Heat (Ruhnau et al. 2019) for the heat-pump efficiency method that would be used if a temperature series were available.